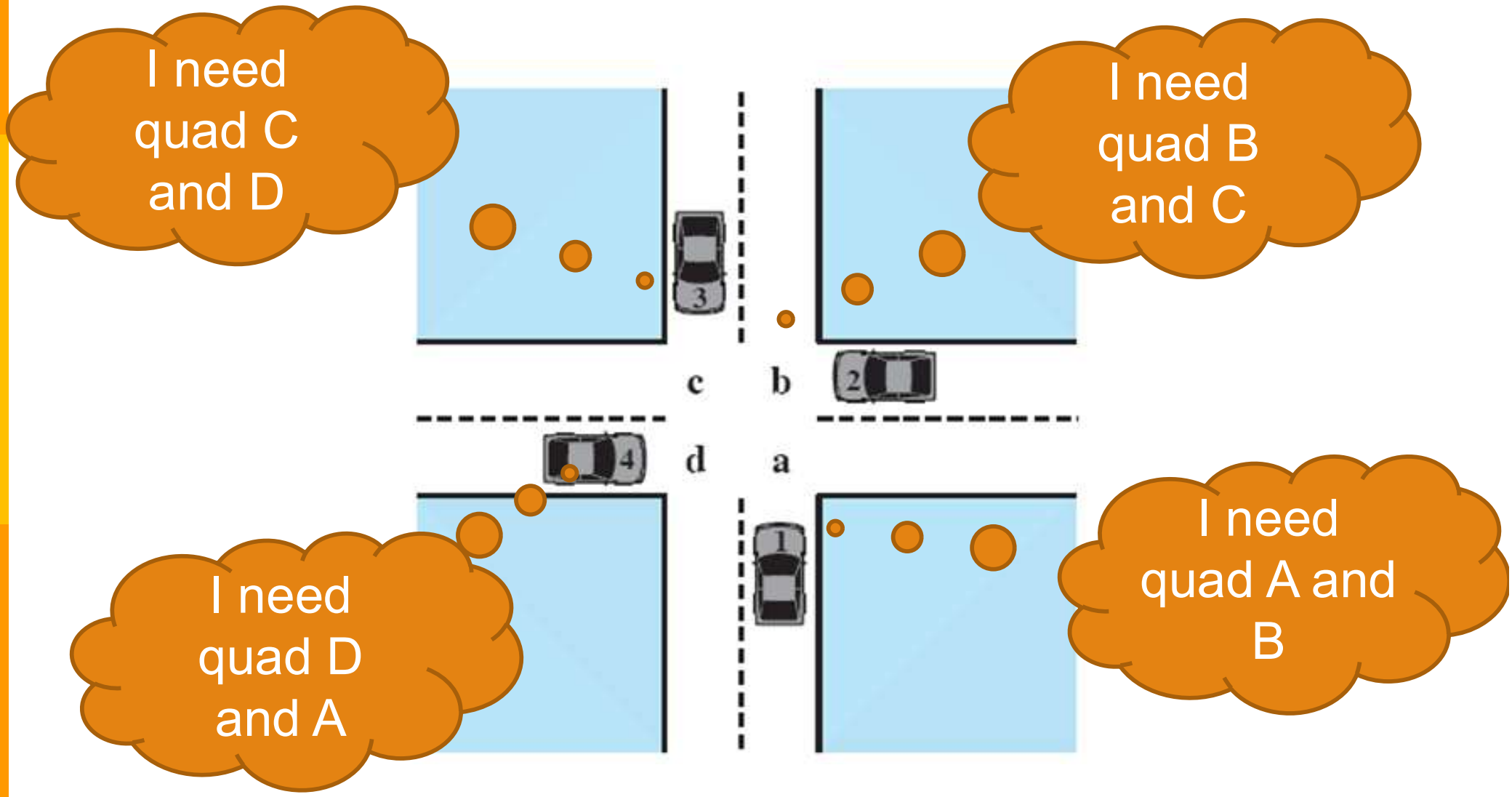
CONCURRENCY

Deadlocks

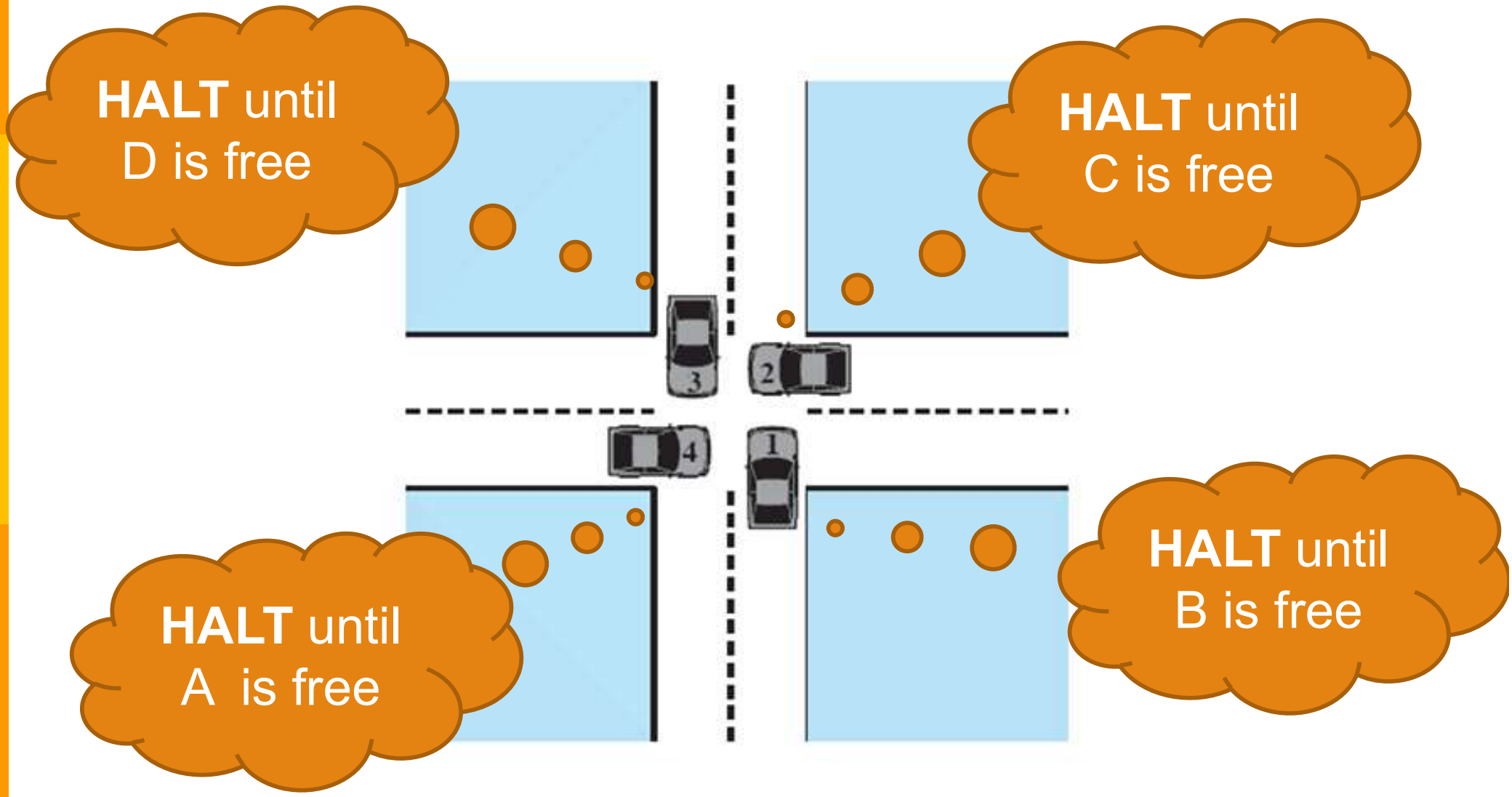
Deadlock

- A set of processes is deadlocked when each process in the set is blocked awaiting an event that can only be triggered by another blocked process in the set
- Typically involves processes competing for the same set of resources
- There is no efficient solution in the general case

Potential Deadlock



Actual Deadlock



Joint Progress Diagram of Deadlock

- Two Processes P and Q
- Each needing exclusive access to a resource A and B for a period of time

Process P	Process Q
• • •	• • •
Get A	Get B
• • •	• • •
Get B	Get A
• • •	• • •
Release A	Release B
• • •	• • •
Release B	Release A
• • •	• • •

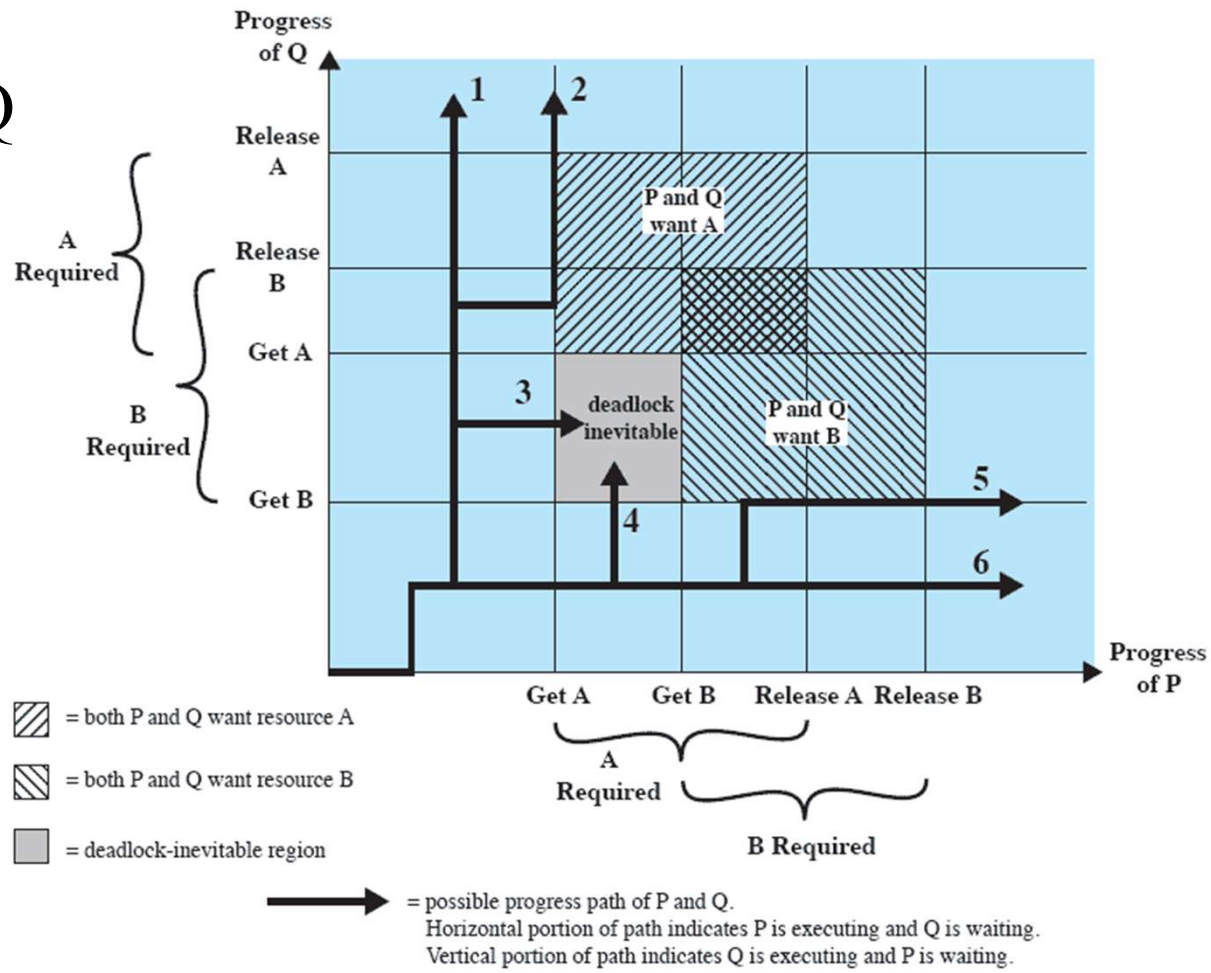


Figure 6.2 Example of Deadlock

Joint Progress Diagram - No Deadlock

- Suppose that P does not need both resources at the same time so that the two processes have this form

Process P	Process Q
• • •	• • •
Get A	Get B
• • •	• • •
Release A	Get A
• • •	• • •
Get B	Release B
• • •	• • •
Release B	Release A
• • •	• • •

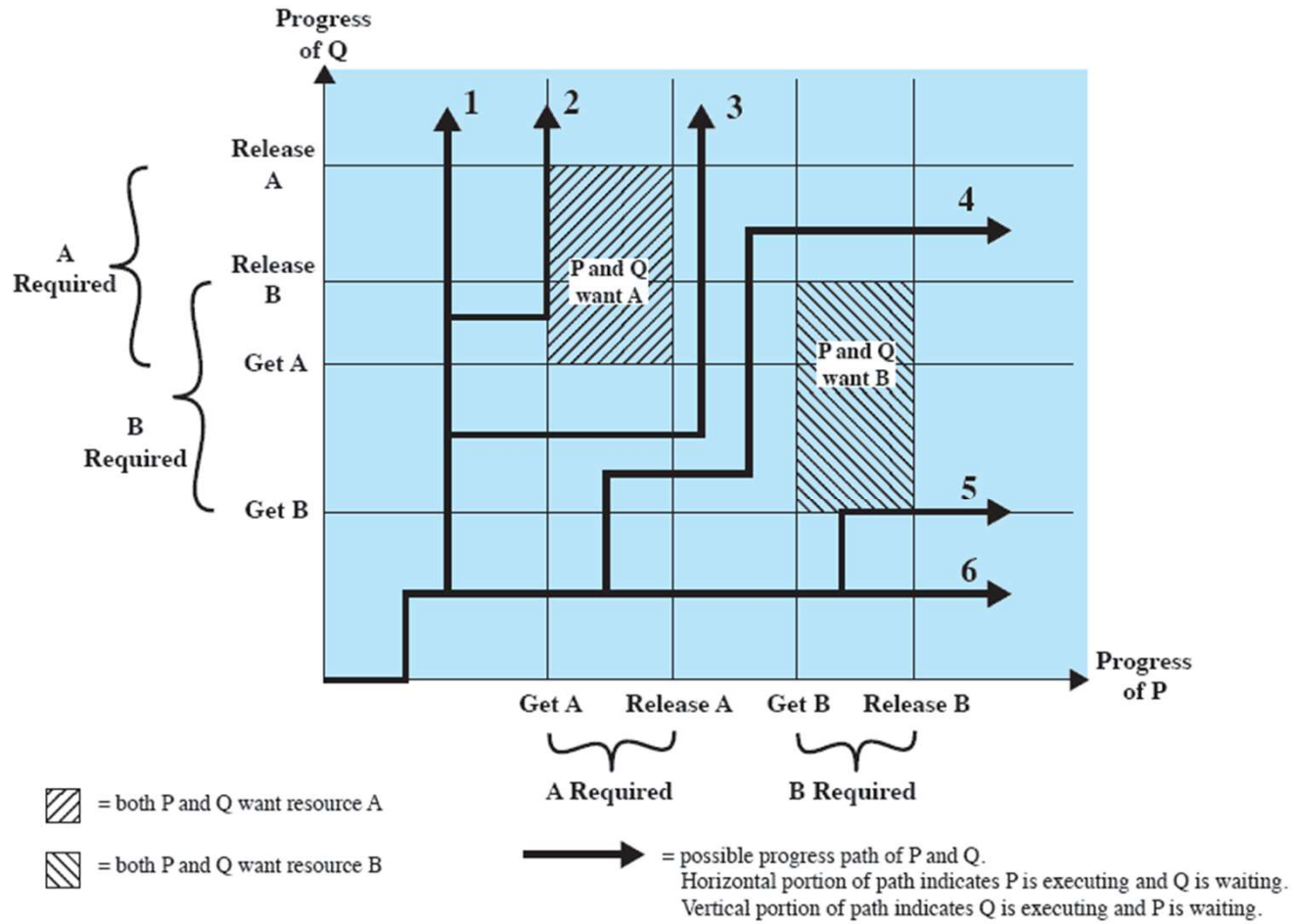


Figure 6.3 Example of No Deadlock [BACO03]

REUSABLE RESOURCES

- Used by one process at a time and not depleted by that use
- Processes obtain resources that they later release for reuse by other processes
- **Ex**: CPU, memory, files, databases, and semaphores
- Deadlock occurs if each process holds one resource and requests the other

CONSUMABLE RESOURCES

- Created (produced) and destroyed (consumed) by a process
- **Ex:** Interrupts, signals, messages, and information in I/O buffers
- Deadlock may occur if a Receive message is blocking

Deadlock Example (reusable resource)

Process P

Step	Action
p_0	Request (D)
p_1	Lock (D)
p_2	Request (T)
p_3	Lock (T)
p_4	Perform function
p_5	Unlock (D)
p_6	Unlock (T)

Process Q

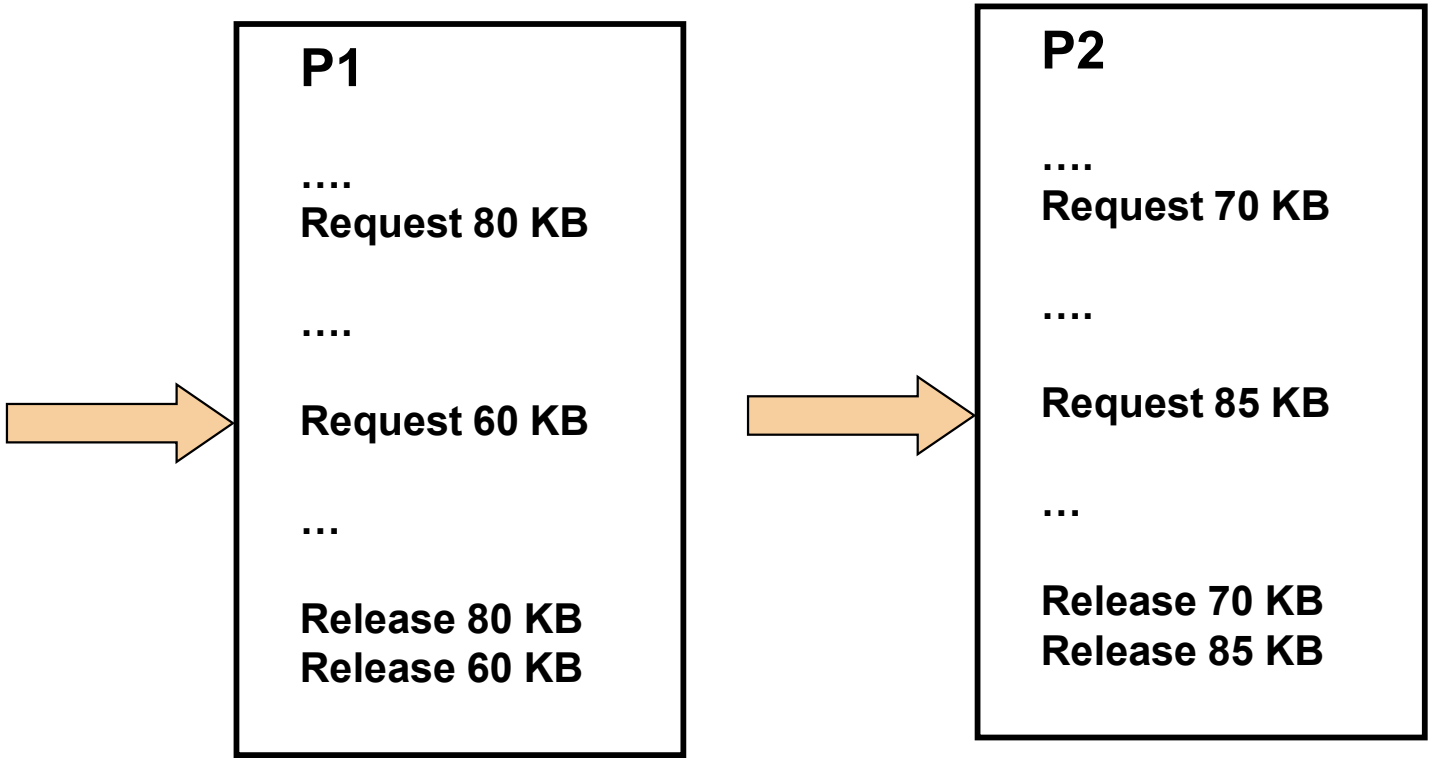
Step	Action
q_0	Request (T)
q_1	Lock (T)
q_2	Request (D)
q_3	Lock (D)
q_4	Perform function
q_5	Unlock (T)
q_6	Unlock (D)

- Two processes P and Q competing for exclusive access to disk file D and tape drive T
- Deadlock occurs if multiprogramming interleaves the execution of P and Q as follows:

$$P_0\ P_1\ Q_0\ Q_1\ P_2\ Q_2$$

Deadlock Example (reusable resource)

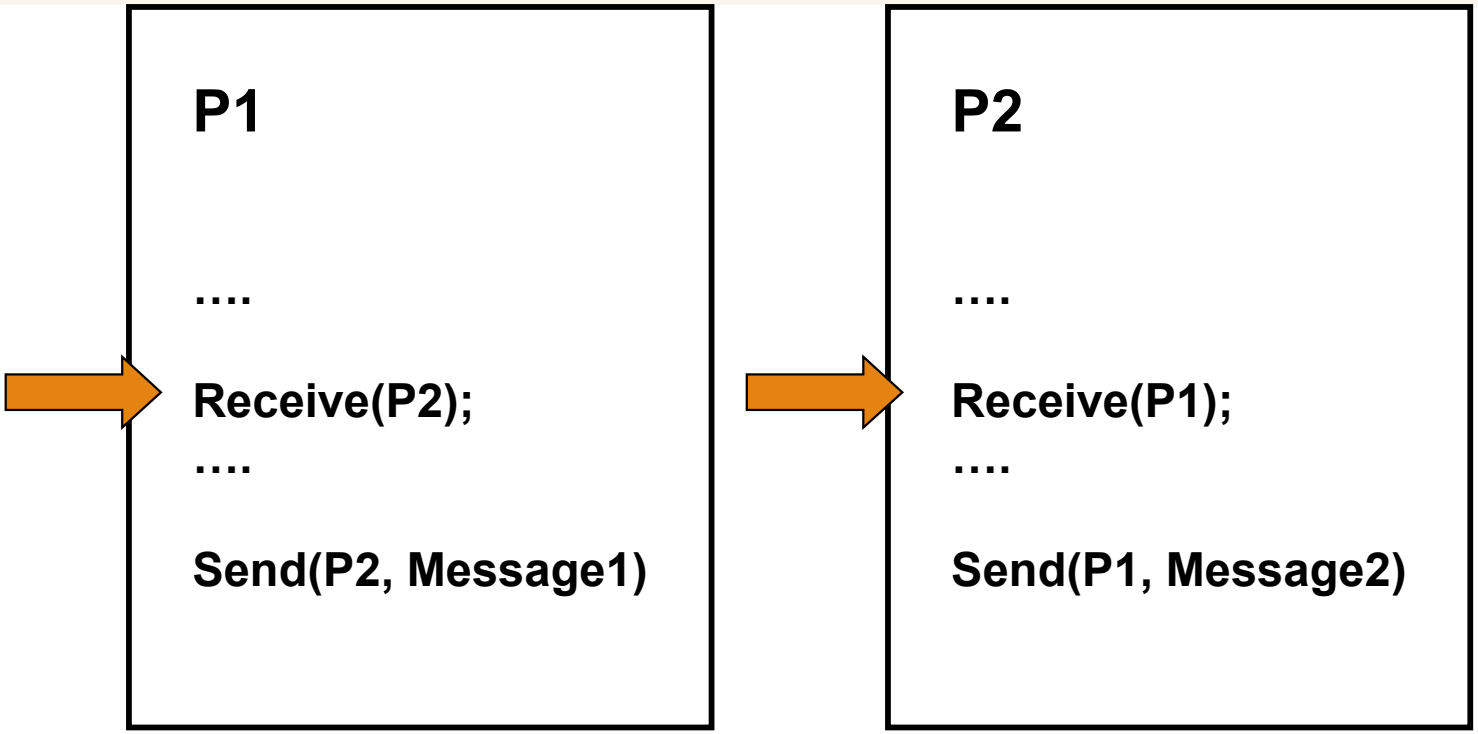
- Space is available for allocation of 200K bytes, and the following sequence of events occur



- Deadlock occurs if both processes progress to their second request
- Cannot be dealt with at system design
- Deal with it using virtual memory

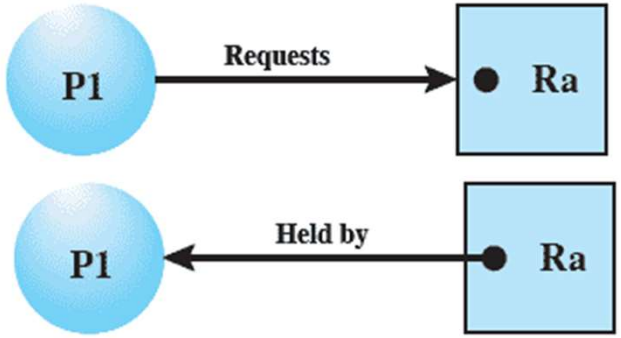
Deadlock Example (consumable resource)

- Consider a pair of processes, in which each process attempts to receive a message from the other process and then send a message to the other process
  - Deadlock occurs if *Receive* is blocking
  - A **design error** is the cause of the deadlock



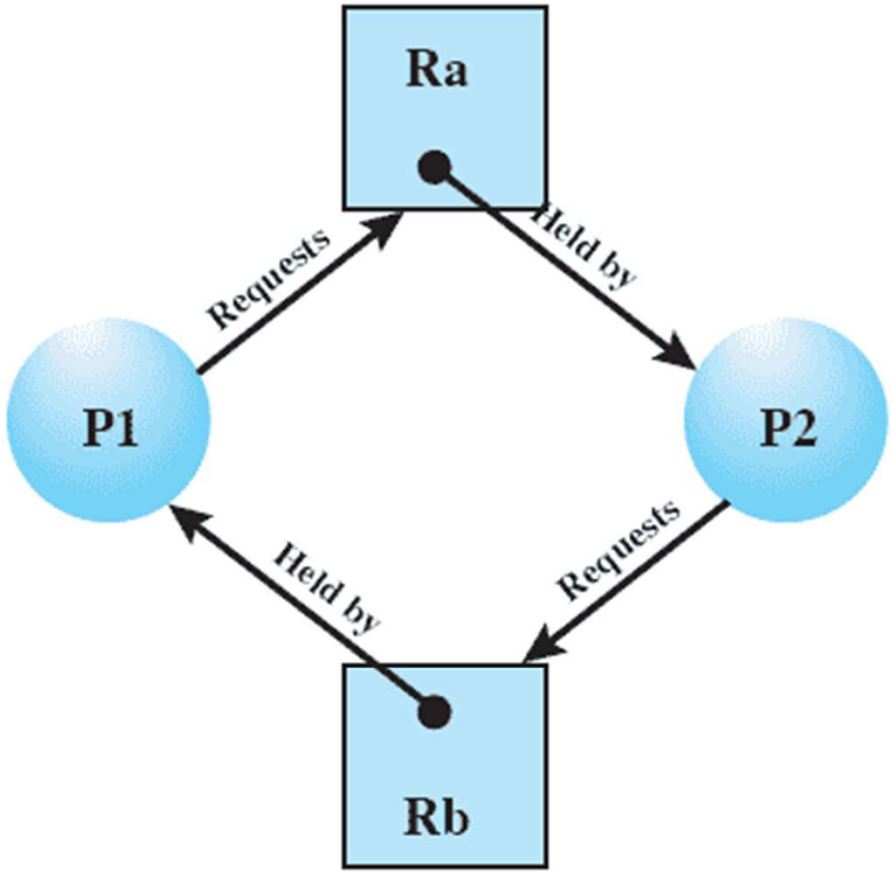
Resource-Allocation Graph

- Deadlocks can be described in terms of a directed graph called: **Resource-Allocation Graph**
- A set of vertices *V* and a set of edges *E*
- V is partitioned into two types:
  - $P = \{P_1, P_2, \ldots, P_n\}$, the set of all the processes in the system
  - $R = \{R_1, R_2, \ldots, R_m\}$, the set of all resource types in the system
- **Request edge** – directed edge $\boldsymbol{P_i \rightarrow R_j}$
- **Assignment edge** – directed edge $\boldsymbol{R_j \rightarrow P_i}$
- Process == circle, Resource == square
- A dot is shown for each instance of that resource

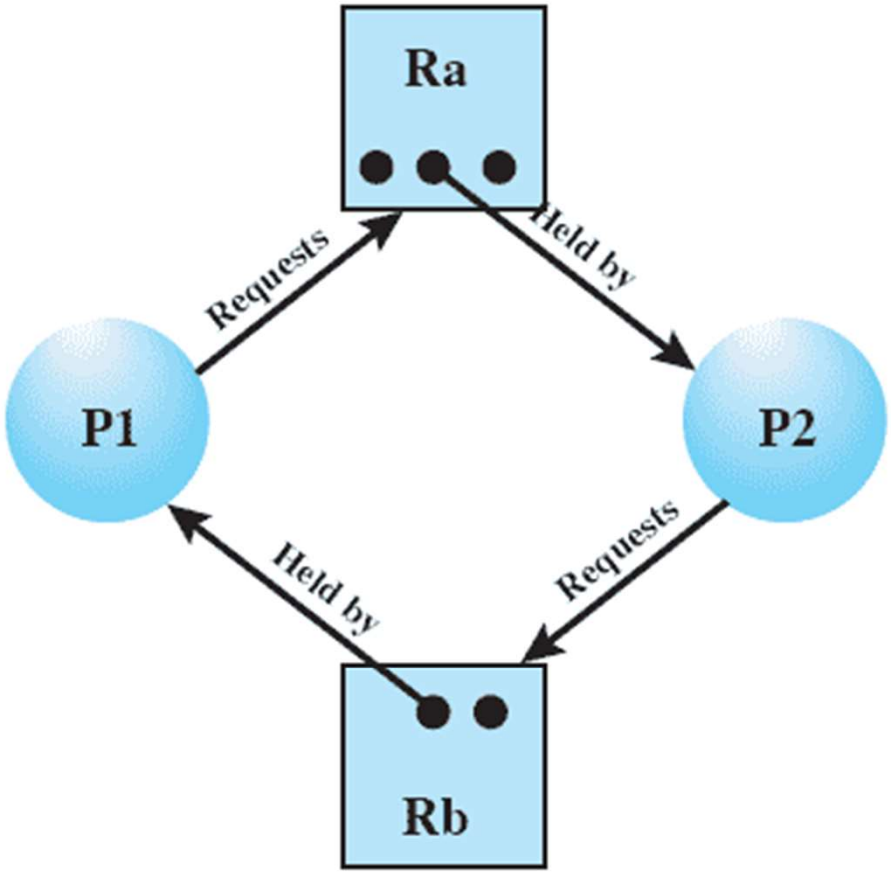


- If the graph contains no cycles, then no process in the system is deadlocked. If the graph does contain a **cycle**, then a **deadlock may exist**.

Resource Allocation Graphs of deadlock



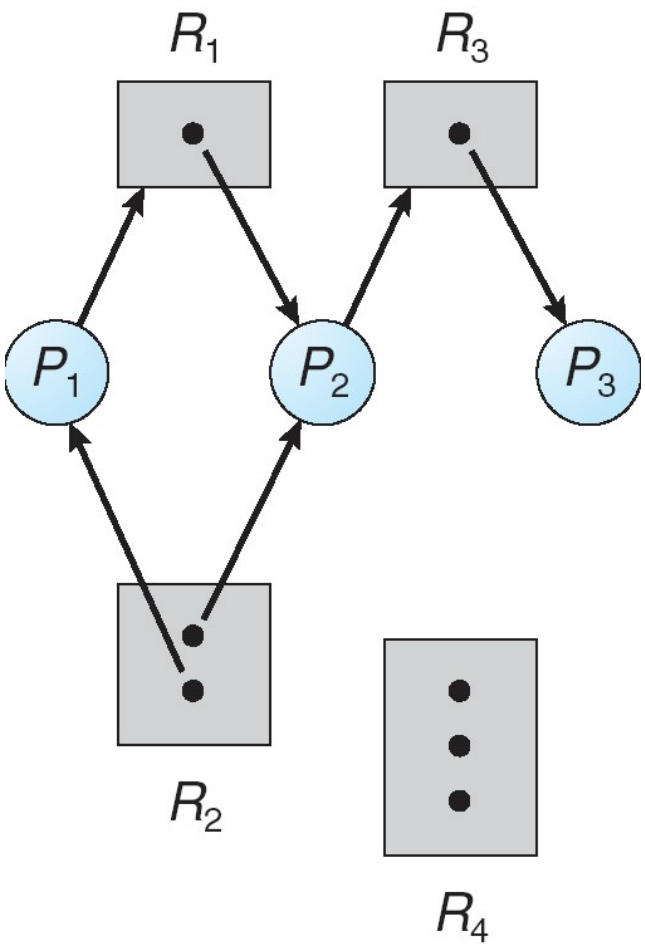
(c) Circular wait



(d) No deadlock

Example of a Resource Allocation Graph

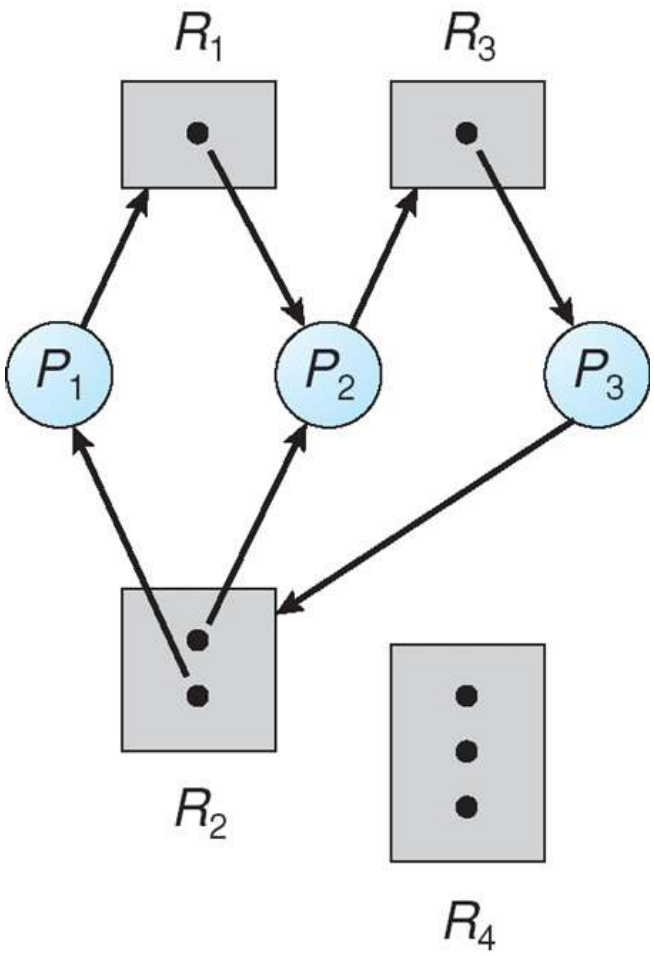
- $P = \{P1, P2, P3\}$
- $R = \{R1, R2, R3, R4\}$
- $E = \{P1 \rightarrow R1, P2 \rightarrow R3, R1 \rightarrow P2, R2 \rightarrow P2, R2 \rightarrow P1, R3 \rightarrow P3\}$



No cycles → No deadlock

Resource Allocation Graph With A Deadlock

P3 requests an instance of R2



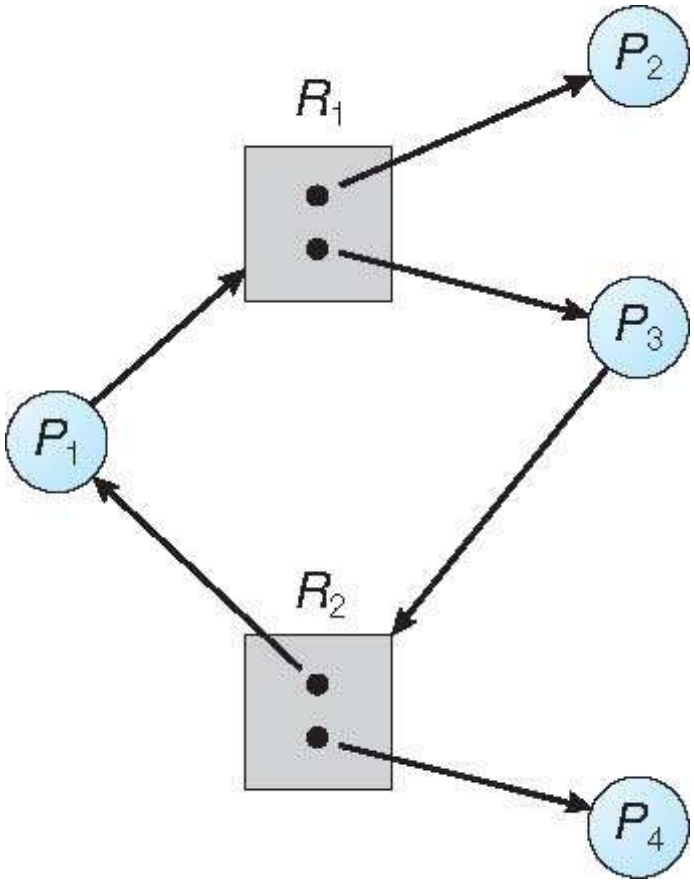
Two cycles exist:

- $P1 \rightarrow R1 \rightarrow P2 \rightarrow R3 \rightarrow P3 \rightarrow R2 \rightarrow P1$
- $P2 \rightarrow R3 \rightarrow P3 \rightarrow R2 \rightarrow P2$

Processes *P*1, *P*2, and *P*3 are **deadlocked**:

- *P*2 is waiting for *R*3, which is held by *P*3.
- *P*3 is waiting for either *P*1 or *P*2 to release *R*2
- *P*1 is waiting for *P*2 to release *R*1.

Graph With A Cycle But No Deadlock

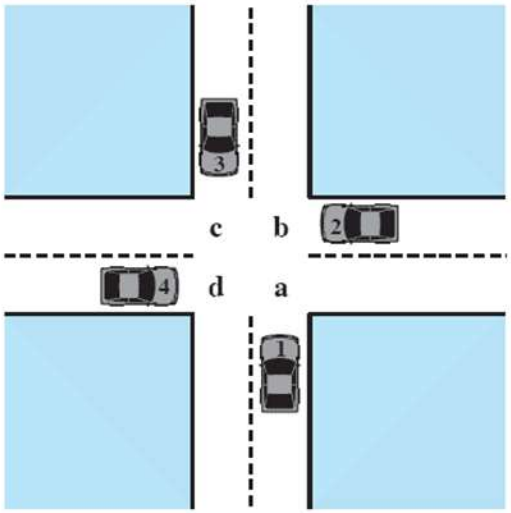


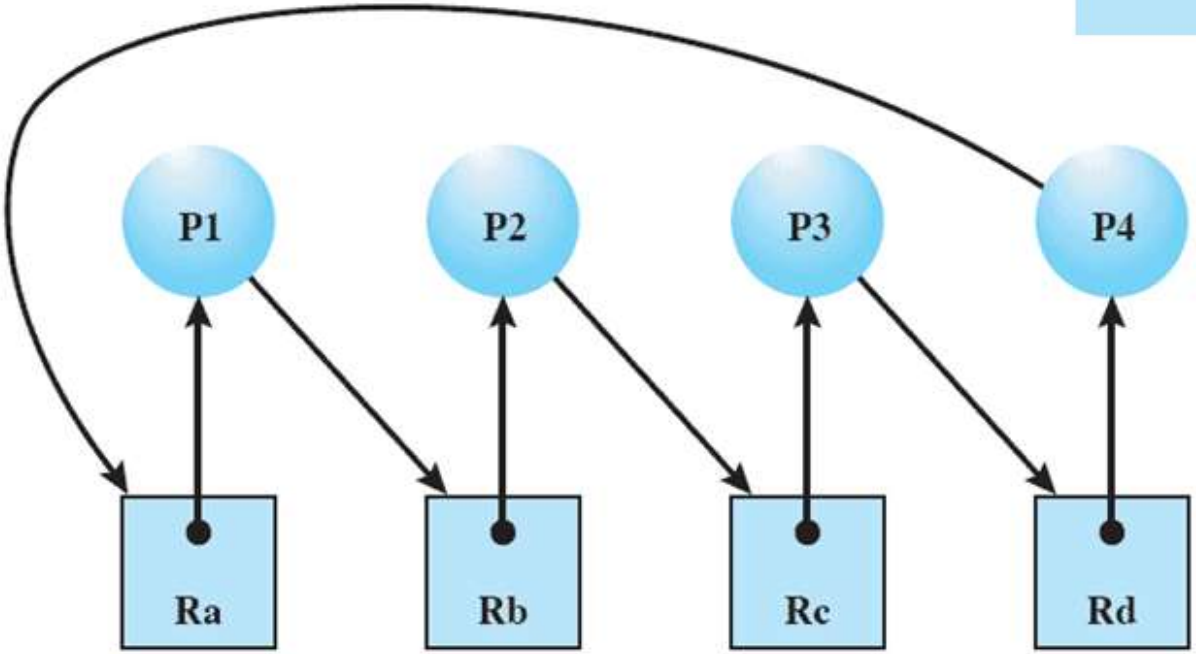
There is a cycle

- P1 → R1 → P3 → R2 → P1

No deadlock

Cars in Intersection, again





Four Conditions for Deadlock

1. Mutual exclusion condition

   Only one process may use a resource at a time

2. Hold-and-wait condition

   A process may hold allocated resources while awaiting assignment of others

3. No preemption condition

   No resource can be forcibly removed form a process holding it

4. Circular wait condition

   A closed chain of processes exists, such that each process holds at least one resource needed by the next process in the chain

- First Three conditions are necessary but not sufficient for deadlock - **all four** conditions must hold for deadlock
- Irresolvable circular wait is the definition of deadlock!

Addressing Deadlock

- ***Prevention*** :
  - Design the system so that deadlock is **impossible**
  - Adopt a policy that eliminates one of the necessary conditions
- ***Avoidance***:
  - Requires advance info on resources required in process lifetime
  - making the appropriate dynamic allocation that, will **not allow** the system to go to a deadlock state
- ***Detection & Recovery***:
  - Allow the system to enter a deadlock state, check for deadlock (periodically or sporadically), then recover
- ***Ignore*** the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX
  - Performance deteriorates and system halts, manual restart
  - Cost: ignoring could be cheaper than above approaches

Deadlock Prevention

Idea: invalidate one of the four conditions for deadlock

1. Mutual exclusion condition
2. Hold-and-wait condition
3. No preemption condition
4. Circular wait condition

- Two main methods:
  - Indirect Prevent the occurrence of one of the three necessary conditions
  - Direct Prevent the occurrence of a circular wait

"Mutual exclusion" condition

- Mutual Exclusion = only one process may use a resource at a time

To prevent deadlock

➔ do not allow "mutual exclusion"

- Not applicable to all resources e.g. Files
  - not required for sharable resources (e.g., read-only files)
  - But must hold for any sharable resource (R/W files, semaphores, ..)

"Hold & wait" condition

- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources

To prevent deadlock → avoid hold and wait!

  - Sol 1: Force a process to request all resources it needs before starting.
    - may not know required resources at start of run
    - Process delayed until all resources are granted
    - Ties up resources other processes could be using
  - Sol 2: If a process needs to acquire a new resource, it must first release all resources it holds, then reacquire all it needs
  - Low resource utilization; starvation possible

"No preemption" condition

To prevent deadlock ➔ allow preemption !

- If a process holding certain resources is denied a further request, that process must release its original resources and, if necessary, request them again together with the additional resource (➔ inefficient !)
  - Consider a process given the printer halfway through its job
- If a process requests a resource that is held by another process, the OS may preempt the second process and require it to release its resources

  (➔ waste of resources)
- Can only be used in very special circumstances
  - resources whose state can be easily saved and restored later e.g. processor

"Circular Wait" condition

For deadlock to occur, there must be a cycle in the graph of processes and resources - **Circular Wait**

To prevent deadlock → do not go into a circular wait!

- impose a ***total ordering*** of all resource types, and require that each process requests resources in an increasing order of enumeration
- A process can only ask for R_x

  if $R_i < R_x$ for $\forall R_i$ the process is currently holding
- Whenever a process requests a resource, it must release all resources that are lower
- Inefficient: slowing down processes and denying resource access unnecessarily

Deadlock Avoidance

- Allow the three necessary conditions, but grant only when ***safe***
- Avoidance allows more concurrency than prevention
- A decision is made dynamically whether the current resource allocation request will, if granted, potentially lead to a deadlock
- Requires knowledge of future process requests
- Two approaches to deadlock avoidance:
  - Process Initial Denial: Do not start a process if its demands might lead to deadlock.
  - Resource Allocation Denial Do not grant an *incremental* resource request to a process if this allocation might lead to deadlock.

Deadlock Avoidance

Consider a system with ***n*** processes and ***m*** different types of resources

[Resource = $\mathbf{R} = (R_1, R_2, \ldots, R_m)$	total amount of each resource in the system
Available = $\mathbf{V} = (V_1, V_2, \ldots, V_m)$	total amount of each resource not allocated to any process
Claim = $\mathbf{C} = \begin{bmatrix} C_{11} & C_{12} & \ldots & C_{1m} \\ C_{21} & C_{22} & \ldots & C_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ C_{n1} & C_{n2} & \ldots & C_{nm} \end{bmatrix}$	C_{ij} = requirement of process i for resource j
Allocation = $\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & \ldots & A_{1m} \\ A_{21} & A_{22} & \ldots & A_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ A_{n1} & A_{n2} & \ldots & A_{nm} \end{bmatrix}$	A_{ij} = current allocation to process i of resource j

The following relationships must hold:

1. $R_j = V_j + \sum_{i=1}^{n} A_{ij}$, for all j — All resources are either available or allocated.
2. $C_{ij} \leq R_j$, for all i, j — No process can claim more than the total amount of resources in the system.
3. $A_{ij} \leq C_{ij}$, for all i, j — No process is allocated more resources of any type than the process originally claimed to need.

Process Initial Denial

- A process is only started if the maximum claim of all current processes plus those of the new process can be met.
- Start a new process P_{n+1} only if

$$R_j \geqslant C_{(n+1)j} + \sum_{i=1}^{n} C_{ij} \quad \text{for all } j$$

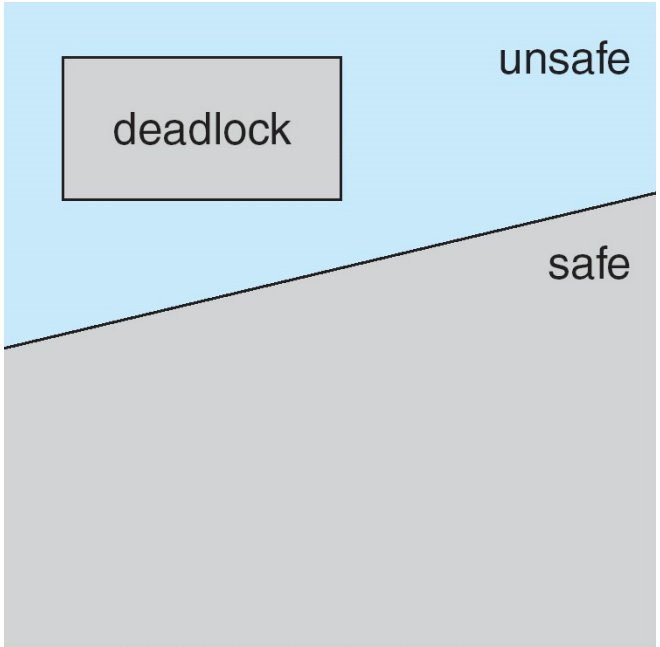
- Track current allocations
- Not optimal - Assumes the worst: that all processes will make their maximum claims <u>at the same time</u>.
- Requires knowledge of future process requests: maximum claim for each process

Safe State

- A state is ***safe*** if the system can allocate resources to each process (up to its max) in some order and still avoid deadlock
- When a process requests an available resource, system must decide if *immediate allocation* leaves the system in a safe state
- System is in **safe state** if there exists a sequence $<P_1, P_2, \ldots, P_n>$ of ALL the processes in the systems such that for each P_i, the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j, with $j < i$
- That is:
  - If P_i resource needs are not immediately available, then P_i can wait until all P_j have finished
  - When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate
  - When P_i terminates, P_{i+1} can obtain its needed resources, and so on
- If no such sequence exists, then the system state is said to be ***unsafe***.

Basic Facts

- If a system is in safe state ⇒ no deadlocks
- If a system is in unsafe state ⇒ **possibility** of deadlock
- Avoidance ⇒ ensure that a system will never enter an unsafe state.
  - Initially, the system is in a safe state
  - Whenever a process requests a resource that is currently available, The request is granted only if the allocation leaves the system in a safe state.
  - if a process requests a resource that is currently available, it may still have to wait.
- Thus, resource utilization may be lower than it would otherwise be.



Resource Allocation Denial

- Consider a system with fixed number of resources
  - ***State*** : current allocation of resources to process (R, C, V, A)
  - ***Safe state:*** there is at least one sequence of resource allocation to processes that does not result in deadlock: all processes can run to completion
  - ***Unsafe state*** is a state that is not safe
- ***Banker's Algorithm*** (Dijkstra)
  - Reject a request if it exceeds the processes' declared maximum claims
  - Grant a request if the new state would be safe
  - Determining if a state is safe
    - Find any process P_i for which we can meet its maximum requests
    - Don't forget already allocated resources
    - Mark P_i as "done", add its resources to available resource pool
    - State is *safe* if we can mark all processes as "done"
  - Block a process if the resources are not currently available or the new state is not safe

Determination of Safe State

- A system consisting of four processes and three resources.
- Allocations are made to processors resulting in R and V
- ***Is this a safe state?***
  - Can any of the four processes be run to completion with the resources available?
  - That is, can the difference between the maximum requirement and current allocation for any process be met with the available resources?

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	2	2	2
P2	0	0	1
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
0	1	1

Available vector **V**

(a) Initial state

Amount of Existing Resources

Resources available after allocation

Process *i*

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	2	2	2
P2	0	0	1
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
0	1	1

Available vector **V**

(a) Initial state

- Can any of the remaining processes can be completed?

  $C_{ij} - A_{ij} \leq V_j$, for all j

- This is not possible for P1,
  - which has only 1 unit of R1 and requires 2 more units of R1, 2 units of R2, and 2 units of R3.
- If we assign one unit of R3 to process P2,
  - Then P2 has its maximum required resources allocated and can run to completion and return resources to ‘available’ pool

Is this state *Safe* ?

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	2	2	2
P2	0	0	1
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
0	1	1

Available vector **V**

(a) Initial state

	R1	R2	R3
P1	3	2	2
P2	0	0	0
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	1	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	2	2	2
P2	0	0	0
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
6	2	3

Available vector **V**

(b) P2 runs to completion

After P2 runs to completion

Is this state *Safe* ?

	R1	R2	R3
P1	3	2	2
P2	0	0	0
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	1	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	2	2	2
P2	0	0	0
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
6	2	3

Available vector **V**

(b) P2 runs to completion

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
7	2	3

Available vector **V**

(c) P1 runs to completion

After P1 completes

Is this state *Safe* ?

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
7	2	3

Available vector **V**

(c) P1 runs to completion

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	0	0	0
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
9	3	4

Available vector **V**

(d) P3 runs to completion

Yes, the given state is SAFE ! Because there is at least one scenario in which all processes can finish: P2, P1, P3, and P4

When should we grant a request ?

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	1	0	0
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	2	2	2
P2	1	0	2
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
1	1	2

Available vector **V**

If P1 requests 1 unit each of R1 and R3, should we grant it?

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	2	0	1
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	1	2	1
P2	1	0	2
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
0	1	1

Available vector **V**

No! Because, there is a possibility that a deadlock may occur!
each process will need at least one more unit of R1; there are none available

Avoidance Example

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix **C**

	R1	R2	R3
P1	2	0	1
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix **A**

	R1	R2	R3
P1	1	2	1
P2	1	0	2
P3	1	0	3
P4	4	2	0

C – A

R1	R2	R3
9	3	6

Resource vector **R**

R1	R2	R3
0	1	1

Available vector **V**

- Are we in a safe state? No!
- Are we deadlocked? No! But, a potential for deadlock.

- It is possible, for example, that if P1 runs, then release one unit of R1 and one unit of R3 prior to needing these resources again. If that happened, the system would return to a safe state
- So, deadlock avoidance strategy does not predict deadlock with certainty; it anticipates the possibility of deadlock and assures that there is never such a possibility.

Deadlock Avoidance Strategy

- When a process makes a request for a set of resources
  - assume that the request is granted
  - Update the system state accordingly
- Then determine if the result is a safe state
  - If so, grant the request
  - if not, block the process until it is safe to grant the request
- Advantages
  - It is not necessary to pre-empt / rollback processes, as in deadlock detection
  - It is less restrictive than deadlock prevention
- Restrictions
  - Maximum resource requirement must be stated in advance
  - Processes must be independent i.e. unordered and with no synchronization requirements
  - There must be a fixed number of resources to allocate
  - No process may exit while holding resources

Deadlock Avoidance Logic

```
struct state {
        int resource[m];
        int available[m];
        int claim[n][m];
        int alloc[n][m];
}
```

(a) global data structures

```
if (alloc [i,*] + request [*] > claim [i,*])
     < error >;                                  /* total request > claim*/
else if (request [*] > available [*])
     < suspend process >;
else {                                                /* simulate alloc */
     < define newstate by:
     alloc [i,*] = alloc [i,*] + request [*];
     available [*] = available [*] - request [*] >;
}
if (safe (newstate))
     < carry out allocation >;
else {
     < restore original state >;
     < suspend process >;
}
```

(b) resource alloc algorithm

Deadlock Avoidance Logic

```
boolean safe (state S) {
    int currentavail[m];
    process rest[<number of processes>];
    currentavail = available;
    rest = {all processes};
    possible = true;
    while (possible) {
        <find a process Pk in rest such that
            claim [k,*] – alloc [k,*] <= currentavail;>
        if (found) {                        /* simulate execution of Pk */
            currentavail = currentavail + alloc [k,*];
            rest = rest - {Pk};
        }
        else possible = false;
    }
    return (rest == null);
}
```

(c) test for safety algorithm (banker's algorithm)

Deadlock Detection

- Deadlock prevention strategies are very conservative
  - limit access to resources and impose restrictions on processes.
- Deadlock detection strategies do the opposite
  - Resource requests are granted whenever possible
  - Regularly check for deadlock
- Deadlock checks can be made at each resource request
  - **++** leads to early detection,
  - **++** the algorithm is relatively simple because it is based on incremental changes to the state of the system.
  - **--** frequent checks consume processor time

Deadlock Detection Algorithm

- Preparation:
  - Create table of process requests, current allocations
  - Note available resources
- Mark processes with no resources
- Mark any process whose requests can be met (requests $\leq$ available resources)
  - Include resources from that process as 'available' (this process can finish)
  - If multiple processes available, pick any
- If any processes cannot be marked, they are part of a deadlock

Deadlock Detection Algorithm

- Use a Allocation matrix ***A*** and Available vector ***V***
- Use a request matrix ***Q*** Q_{ij} = amount of resource ***j*** requested by process ***i***
- 'un-mark' all processes that are not deadlocked (Initially all processes)

1. Mark each process that has a row in ***A*** of all zeros.
2. Initialize a temporary vector **W** to equal the Available vector.
3. Find a process ***i*** currently unmarked and the ith row of $Q \leq W$.
   - i.e. $Q_{ik} \leq W_k$ for $1 \leq k \leq m$.
   - If no such row is found, terminate
4. If such a row is found,
   - mark process *i* and add the corresponding row of the allocation matrix to ***W***.
   - i.e. set $W_k = W_k + A_{ik}$, for $1 \leq k \leq m$
   - Return to step 3.

- A deadlock exists iff there are unmarked processes at the end
- Each unmarked process is deadlocked.

Deadlock Detection

1. Mark P4 as "not deadlocked" since P4 has no allocated resources
2. Available vector W= (0 0 0 0 1)
3. P3's request vector ≤ (0 0 0 0 1)
   so mark P3 as "not deadlocked", and release its resources
   Now, W=W + (0 0 0 1 0)= (0 0 0 1 1)
4. Unfortunately, no other process has a row ≤ Available vector
5. Therefore ====> P1 and P2 are deadlocked.

	R1	R2	R3	R4	R5
P1	0	1	0	0	1
P2	0	0	1	0	1
P3	0	0	0	0	1
P4	1	0	1	0	1

Request Matrix Q

	R1	R2	R3	R4	R5
P1	1	0	1	1	0
P2	1	1	0	0	0
P3	0	0	0	1	0
P4	0	0	0	0	0

Allocation Matrix A

R1	R2	R3	R4	R5
2	1	1	2	1

Resource Vector

R1	R2	R3	R4	R5
0	0	0	0	1

Available Vector

Detection Example 1

Request

	A	B	C	D	E
P_1	0	1	0	0	1
P_2	0	0	1	0	1
P_3	0	0	0	0	1
P_4	1	0	1	0	1

Allocation

	A	B	C	D	E
P_1	1	0	1	1	0
P_2	1	1	0	0	0
P_3	0	0	0	1	0
P_4	0	0	0	0	0

Resource

A	B	C	D	E
2	1	1	2	1

Available

A	B	C	D	E
0	0	0	0	1

Temporary Available

A	B	C	D	E
0	0	0	1	1

- Mark P_4 (not holding anything someone else wants)
- Mark P_3, new available: 0 0 0 1 1
- Cannot mark P_1 or P_2, so we are deadlocked

Detection Example 2

Requests

	A	B	C
P_1	0	0	0
P_2	2	0	2
P_3	0	0	0
P_4	1	0	0
P_5	0	0	2

Allocation

	A	B	C
P_1	0	1	0
P_2	2	0	0
P_3	3	0	3
P_4	2	1	1
P_5	0	0	2

Resource

A	B	C
7	2	6

Available

A	B	C
0	0	0

Temporary Available

A	B	C
7	2	6

- Are we deadlocked?
  - No!

Detection Example 3

Request

	A	B	C	D	E
P_1	0	1	0	0	0
P_2	0	0	1	0	0
P_3	0	0	0	1	0
P_4	0	0	0	0	1
P_5	1	0	0	0	0

Allocation

	A	B	C	D	E
P_1	1	0	0	0	0
P_2	0	1	0	0	0
P_3	0	0	1	0	0
P_4	0	0	0	1	0
P_5	0	0	0	0	1

Resource

A	B	C	D	E
1	1	1	1	1

Available

A	B	C	D	E
0	0	0	0	0

Temporary Available

A	B	C	D	E
0	0	0	0	0

- Cannot mark P_1 thru P_5, so we are deadlocked

Detection Example 4

Request

	A	B	C	D	E
P_1	0	1	0	0	0
P_2	0	0	0	0	0
P_3	0	0	1	0	0
P_4	0	0	0	1	1
P_5	1	0	0	0	0

Allocation

	A	B	C	D	E
P_1	1	0	0	0	0
P_2	0	1	1	0	0
P_3	0	0	0	1	0
P_4	0	0	0	0	0
P_5	0	0	0	0	1

Resource

A	B	C	D	E
1	1	1	1	1

Available

A	B	C	D	E
0	0	0	0	0

Temporary Available

A	B	C	D	E
1	1	1	1	1

- Mark P_4, (nothing allocated)
- Mark P_2 , P_1, P_3 , P_5

Detection-Algorithm Usage

- When, and how often, to invoke depends on:
  - How ***often*** a deadlock is likely to occur?
  - How ***many*** processes will need to be rolled back?
- Invoke Detection Algorithm
  - At Every request
    - Considerable overhead
  - Periodically (once per hour, when CPU Utilization <40%)
- Recovery Strategies once deadlock is detected
  - Abort one or more processes to break the circular wait.
  - Preempt some resources from one or more of the deadlocked processes

Recovery from Deadlock: Process Termination

- Two approaches can be adopted
- **Abort all deadlocked processes**
  - High cost: some processes might have computed for a long time and nearly done.
- **Abort one process at a time** until the deadlock cycle is eliminated
  - After each process is aborted, a deadlock detection algorithm must be invoked
  - **Which process** we choose to abort? Minimum cost?
    1. Priority of the process
    2. How long process has computed, and how much longer to completion
    3. Resources the process has used
    4. Resources process needs to complete
    5. How many processes will need to be terminated
    6. Is process interactive or batch?
- Aborting a process is no straightforward e.g. if in middle of updating a file, the file might be left in an inconsistent state

Recovery from Deadlock: Resource Preemption

- Preempt some resources from processes and give them to other processes until deadlock cycle is broken. **Issues**:
- Which resources and processes to be preempted? **Selecting a victim**
  - minimize cost – criteria: No held resources, amount of time the process has consumed, etc ..
- If we preempt a resource from a process, what should be done with that process?
  - **Total Rollback**: abort the process then restart it
    - Not efficient
  - **Rollback** – return to some safe state, restart process for that state
    - Need system to keep more info about state of running processes
- How can we guarantee that resources will not always be preempted from the same process? **Starvation** – same process may always be picked as victim
  - A process can be picked a finite number of times
  - include number of rollback in cost factor

Adv and Disadv

Approach	Resource Allocation Policy	Different Schemes	Major Advantages	Major Disadvantages
Prevention	Conservative; undercommits resources	Requesting all resources at once	•Works well for processes that perform a single burst of activity •No preemption necessary	•Inefficient •Delays process initiation •Future resource requirements must be known by processes
		Preemption	•Convenient when applied to resources whose state can be saved and restored easily	•Preempts more often than necessary
		Resource ordering	•Feasible to enforce via compile-time checks •Needs no run-time computation since problem is solved in system design	•Disallows incremental resource requests
Avoidance	Midway between that of detection and prevention	Manipulate to find at least one safe path	•No preemption necessary	•Future resource requirements must be known by OS •Processes can be blocked for long periods
Detection	Very liberal; requested resources are granted where possible	Invoke periodically to test for deadlock	•Never delays process initiation •Facilitates online handling	•Inherent preemption losses

Mixed Strategy

- May group resources into classes, have a different deadlock strategy for each class
  - Swap Space
    - Prevent deadlocks by requiring all space to be allocated at once
    - Avoidance also possible
  - Tapes/Files
    - Avoidance can be effective here: processes can declare ahead of time the resources required
    - Prevention by ordering resources also possible
  - Main Memory
    - Prevention by Preemption a good approach
  - Internal Resources (channels, etc.)
    - Prevention by ordering resources
- Can use linear ordering between classes